BidForge - Complete Application Documentation

By Nandeesh Kantli
RV University
1RVU22CSE109
nandeeshak.btech22@rvu.edu.in | nkantli24@gmail.com

This is the master product + technical document for the current BidForge build.

Terminology used in this document:

- `RFQ Owner` = procurement owner who creates and manages RFQs.
- `Bidder` = supplier/carrier who participates in auctions.

Deployment (Current)

- Frontend hosting: Vercel
- Frontend live URL: https://bid-forge-nandeeshk.vercel.app/
- Backend hosting: Render

1) Product Overview

1.1 Problem

Procurement teams need transparent price discovery with strict close control, while suppliers need fair live ranking and quick bid revisions.

1.2 Solution

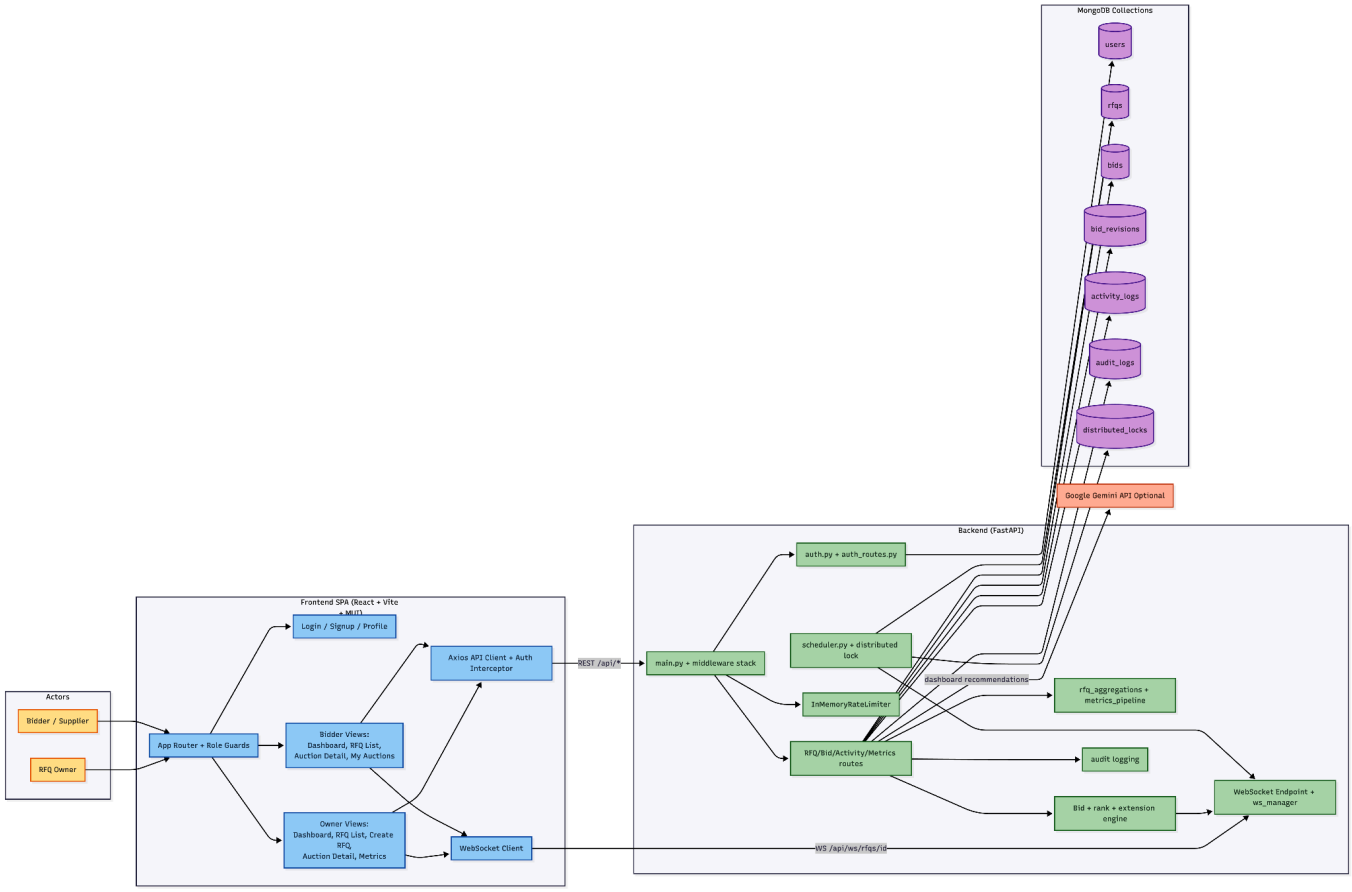
BidForge provides:

- Role-based auction workspace (`rfqowner`, `bidder`).
- Reverse-auction RFQ lifecycle with start/close/forced-close windows.

- British-style extension logic with configurable triggers.
- Real-time updates (status, bid rank, extension events).
- Awarding, exports, timeline, and analytics.

---

2) System Architecture



---

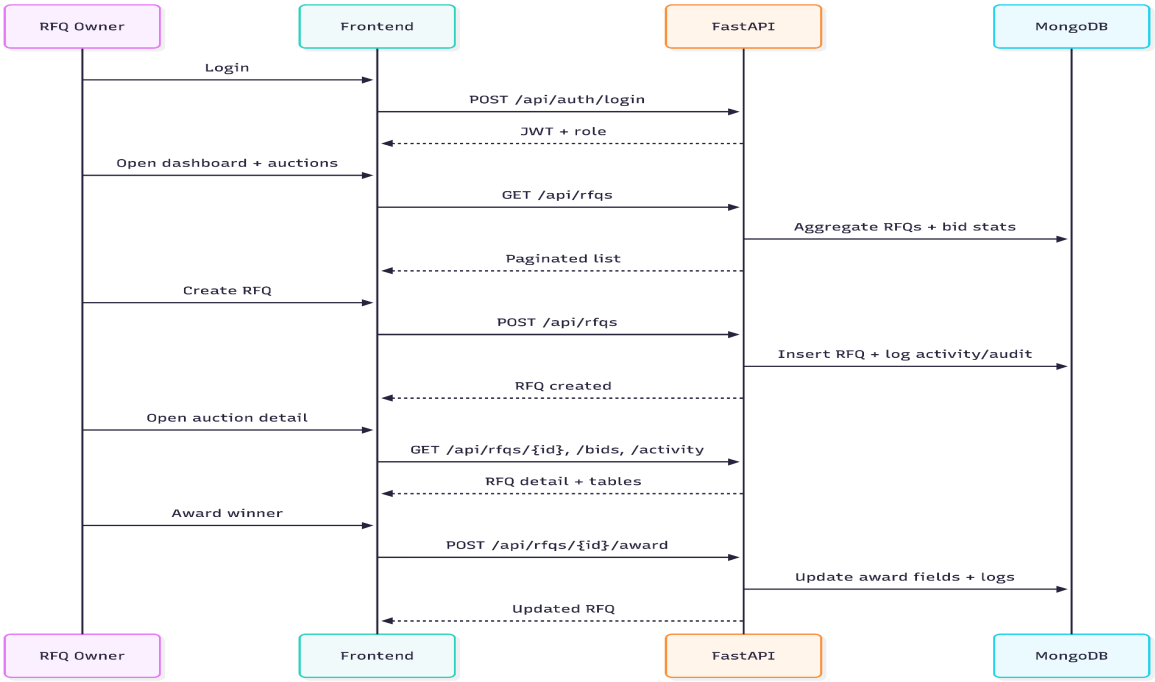
3) End-to-End Role Flows

3.1 RFQ Owner (Buyer) Flow

Screen flow

1. `Login/Signup` -> stores token and role in local storage.
2. `Dashboard` -> sees total/active/upcoming/closed RFQ cards, close-time risk, and recommendations.
3. `Auctions (RFQ List)` -> filters, search, sort, pagination, and extension-aware notifications.
4. `Create RFQ` -> enters timeline, extension config, visibility mode, starting/decrement pricing, and optional file metadata.
5. Redirect to `Auction Detail` after creation.
6. On `Auction Detail`, can:
    - edit auction (when allowed),
    - pause auction (restricted window),
    - export bids/activity CSV,
    - monitor bids and activity timeline,
    - award winner after closure.
7. `Metrics` -> chart-based analytics and paginated metric tables.

API flow



3.2 Bidder / Supplier Flow

Screen flow

1. `Login/Signup` as bidder.
2. `Dashboard` -> sees participated auctions, active bids, L1 rate, Top-3 rate, and risk insights.
3. `Auctions (RFQ List)` -> browse active/upcoming/closed auctions with live countdown.
4. `Auction Detail`:

    submit/revise bid while active,

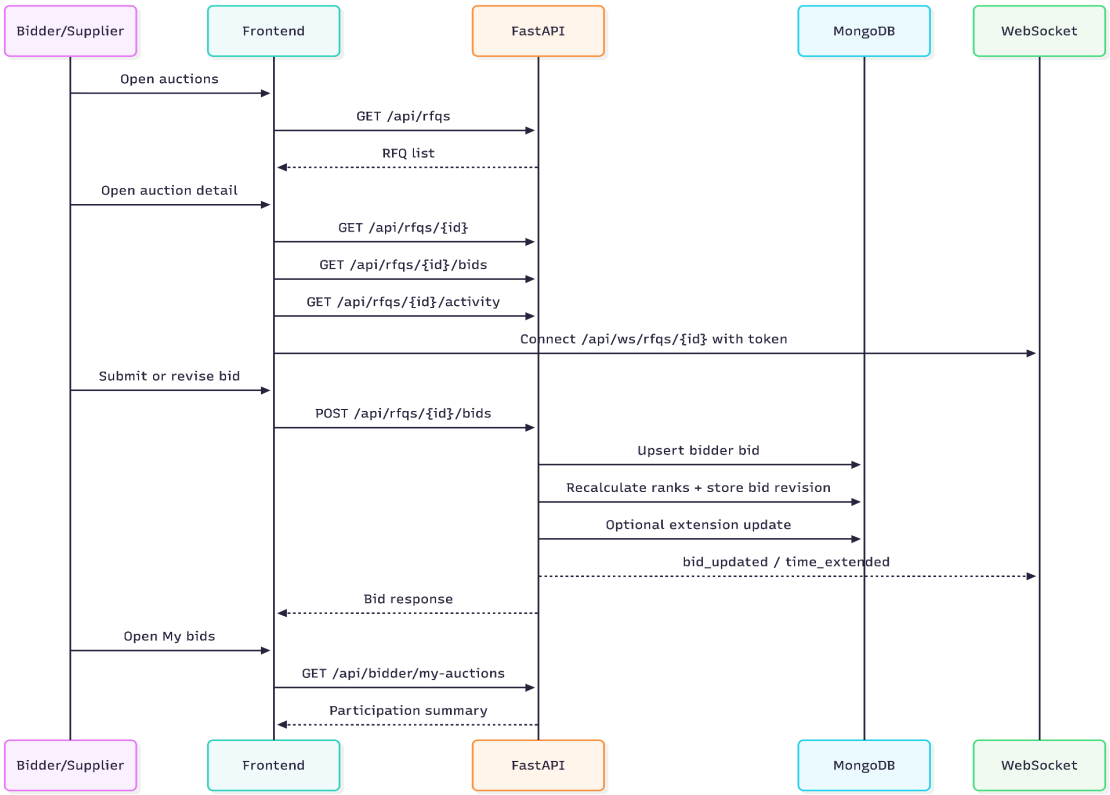
    see own quote details,

    competitor data is masked as per role rules,

    receive live updates over WebSocket.

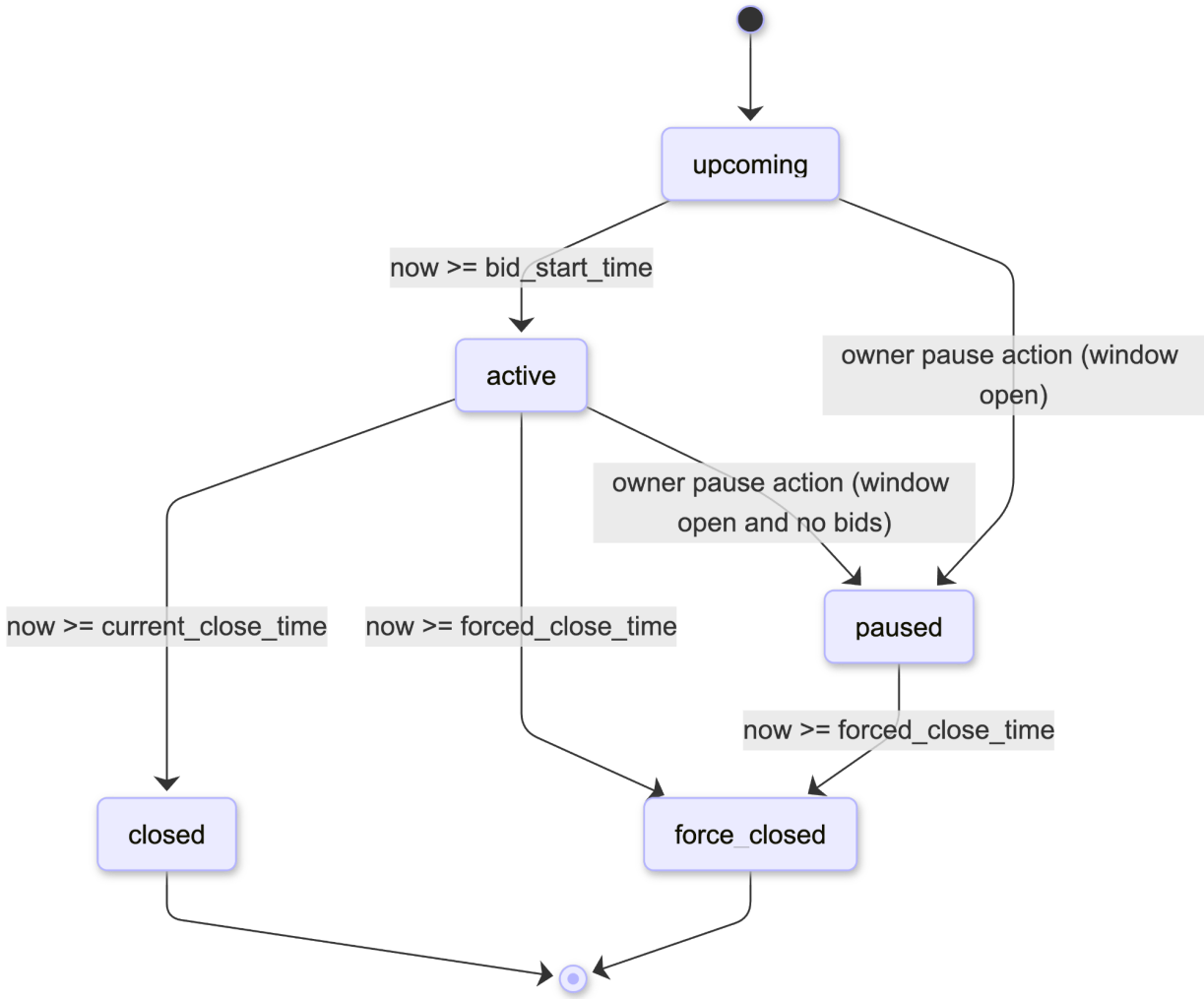
5. `My bids` page -> focused view of auctions where bidder has participated, with rank and L1 comparison.

API flow



---

4) Core Auction Lifecycle



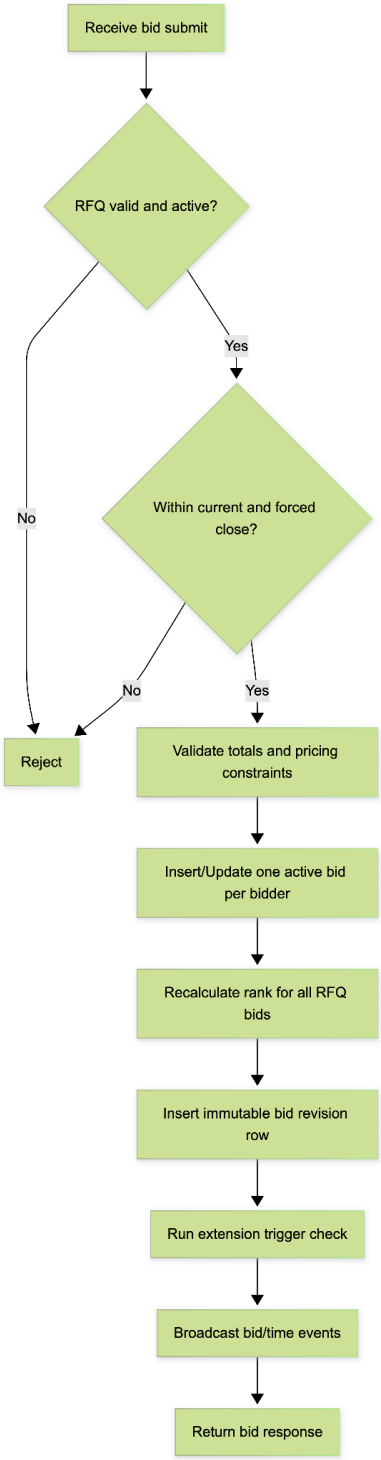
Notes:

- Scheduler persists status transitions periodically.
- Detail/list endpoints also compute status in real-time.

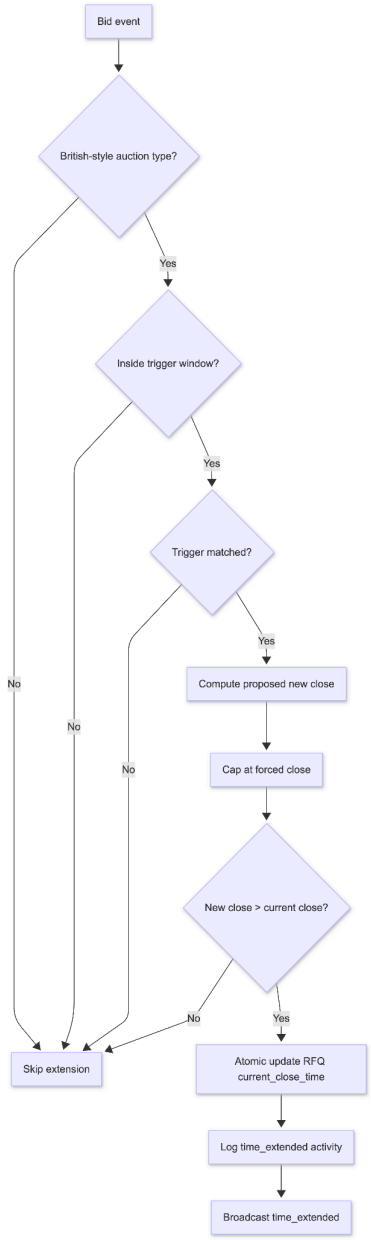
---

5) Bid and Extension Engine

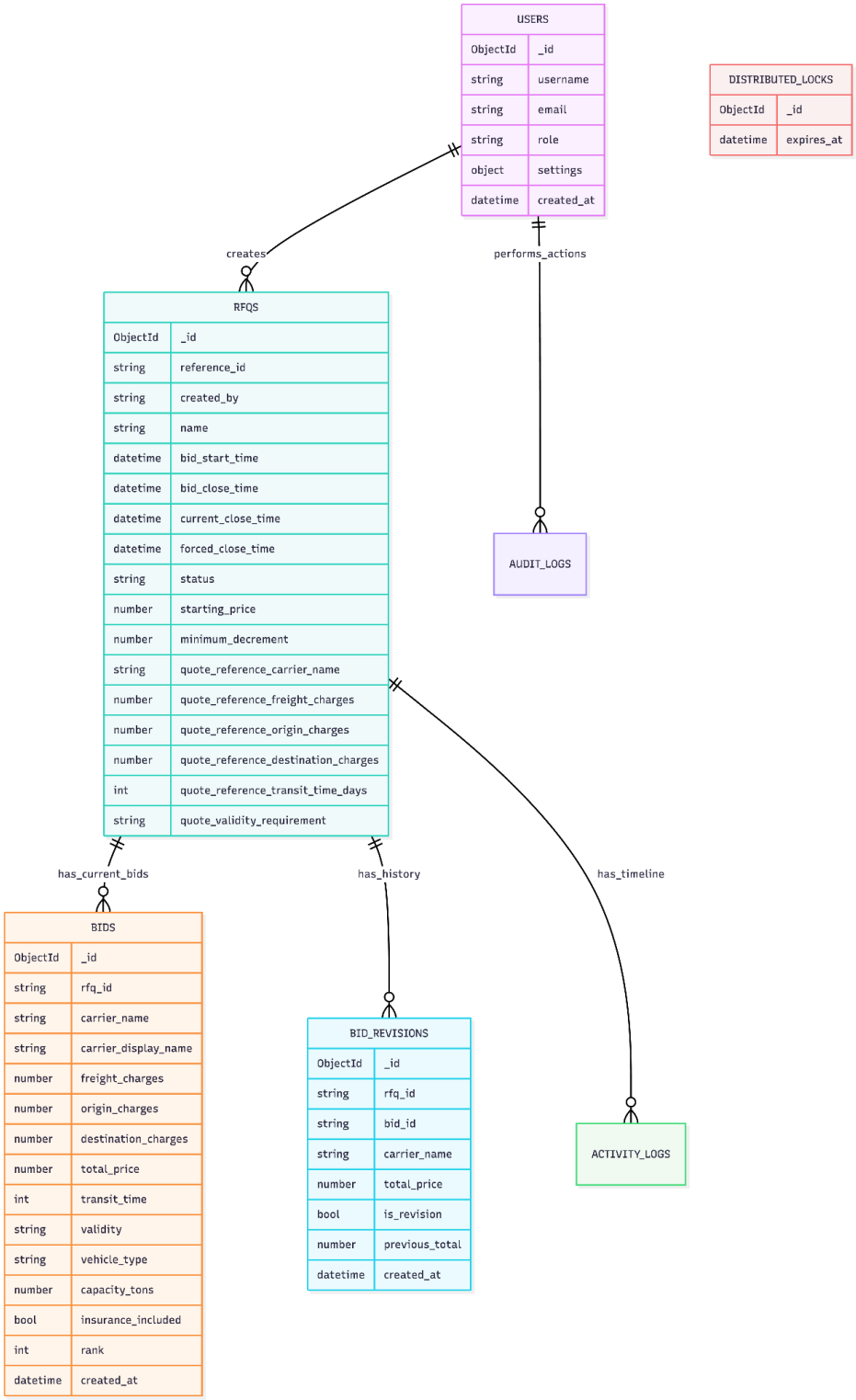
5.1 Bid submit/revise logic


Receive bid submit
RFQ valid and active?
No
Yes
Within current and forced close?
No
Yes
Reject
Validate totals and pricing constraints
Insert/Update one active bid per bidder
Recalculate rank for all RFQ bids
Insert immutable bid revision row
Run extension trigger check
Broadcast bid/time events
Return bid response

5.2 Extension logic



6) Data Model (ER)



7) Frontend Module Mapping

- `Dashboard`:
  - owner: RFQ participation, close pressure, recommendations.
  - bidder: rank health, top-3 rate, risky closes.
- `RFQList`:
  - auto-refresh, search/filter/sort, status cards, countdown.
- `CreateRFQ`:
  - time and pricing validations before submit.
- `AuctionDetail`:
  - bids table, timeline, websocket updates, edit/pause/award controls.
- `SupplierMyAuctions`:
  - bidder participation tracker with countdown and rank.
- `BuyerMetrics`:
  - trend charts and paginated metric tables.

---

8) API Surface

Auth

- `POST /api/auth/signup`
- `POST /api/auth/login`
- `GET /api/auth/me`
- `GET /api/auth/settings`
- `PATCH /api/auth/settings`

RFQ + Auction

- `POST /api/rfqs`
- `GET /api/rfqs`
- `GET /api/rfqs/{rfq_id}`
- `PATCH /api/rfqs/{rfq_id}`
- `DELETE /api/rfqs/{rfq_id}`
- `POST /api/rfqs/{rfq_id}/pause`
- `POST /api/rfqs/{rfq_id}/award`
- `GET /api/bidder/my-auctions`

Bids + Activity + Export

- `POST /api/rfqs/{rfq_id}/bids`
- `GET /api/rfqs/{rfq_id}/bids`

- `GET /api/rfqs/{rfq_id}/bids/export`
- `GET /api/rfqs/{rfq_id}/bid-revisions`
- `GET /api/rfqs/{rfq_id}/activity`
- `GET /api/rfqs/{rfq_id}/activity/export`

Metrics + Recommendations

- `GET /api/metrics/bids-per-rfq`
- `GET /api/metrics/avg-bids`
- `GET /api/metrics/winning-price-trend`
- `GET /api/metrics/extensions-per-rfq`
- `GET /api/metrics/extension-impact`
- `POST /api/dashboard/recommendations`

Realtime

- `WS /api/ws/rfqs/{rfq_id}` with subprotocol token auth.

---

9) Security and Reliability

- JWT bearer auth and role guards.
- PBKDF2 password hashing with legacy bcrypt verify support.
- In-memory rate limits (`global` + stricter `bid submit`).
- Request correlation (`x-request-id`).
- HTTPS enforcement and HSTS in production.
- Scheduler status sync every few seconds with lock control.
- Audit logging for security-sensitive actions.

---

10) Testing and Validation

Backend test focus:

- auction logic
- auth and role checks
- route hardening
- websocket auth

Frontend test focus:

- auth screens

- auction detail behavior
- metrics/dashboard rendering

Manual checklist:

- owner creates RFQ, edits, pauses, awards.
- bidder submits first and revised bids.
- extension triggers near close are applied.
- exports download correctly.
- winner + timeline + metrics are consistent.

---

11) Future Improvements

- distributed websocket/pub-sub (Redis).
- distributed rate limiter.
- stronger file storage integration for specs.
- richer bidder anonymization policies.
- notification channels for close/award events.